

Media Processing Web Application

Cloud Deployment on Amazon Web Services

Project Overview

In this project, we developed the media processing web application which enables users to upload image & video files, with automated pipelines like resizing and watermarking and fetching images through responsive web interface. Our objective is to design, implement and deploy a scalable, production grade distributed system on AWS applying cloud computing principles studied throughout the courses.

Media Processing is a computationally intensive task which is not suited for single server & synchronous architecture. Implementing with HTTP server which processes files inline and block response until the completion is poor user experience which fails to scale. We address the problem through asynchronous, event-driven architecture in which image processing tasks are decoupled from upload requests. This enables the system to handle concurrent jobs without degrading responsiveness.

Our application enables file uploading via a Next.js browser interfaced with immediate acknowledgement, then asynchronous job querying through Redis (EKS Redis Deployment). We implemented a pillow for image operations for media processing and store metadata in PostgreSQL (Amazon Aurora Serverless) and Object Storage for Amazon S3 for both raw uploads and processed outputs. We can access the live deployment over the internet via KuberNet's Load Balancer endpoint on Amazon EKS.

System Design and Technology Stack

This system incorporates microservices architecture which is handled by discrete independently deployable components.

Architectural Overview:

- Frontend(Next.js): The browser facing React Next.js application which includes upload forms, polls, the API for job status and renders the processed output gallery which communicates via FastAPI backend over HTTP.
- API Layer (FastAPI): It is a stateless python HTTP server which receives multipart files uploads, writes jobs records to PostgreSQL, enqueues job identifiers into Redis queue and returns a Job ID to frontend.
- Worker Layer: A long-running Python process that subscribes to the Redis queue. Upon receiving a job it retrieves the source file from S3, performs the configured processing pipeline (Pillow resize and watermark, or FFmpeg transcoding), uploads the result to S3, and updates the job record in PostgreSQL with the output URL and final status
- Data Layer (PostgreSQL in Aurora Serverless): Here we store all job metadata. Managed with Alembic migrations for reproducible database evolution across environments.

- Queue & Cache Layer (Redis via EKS Redis Deployment): Message broker between worker and API which implement consumer-producer pattern where queue name is configurable with environment variable
- Asynchronous processing through a job queue: The system immediately enqueues a lightweight job descriptor and returns, rather than processing files synchronously in the API handler. This design allows the worker tier to be scaled independently of the API tier and ensures that the API will be responsive under load.

Design Decision

Shared codebase for API + Worker: The API and Worker containers are built from the same Docker image (media-tool-api) and make use of all Python modules. The worker is started with a different entry point (python -m app.worker.runner). This eases dependency management and keeps the library versions consistent between the two services - a conscious tradeoff of size for simplicity of operation.

S3 as a storage backend: Local filesystem storage is incompatible with horizontally scaled, multi-node deployments. S3 provides a network addressable, durable object store accessible from random pods. Through LocalObjectStorage, the storage layer is abstracted which makes the backend switchable through the STORAGE_BACKEND environment variable.

RDS for PostgreSQL: Our AWS Academy course had limitations on available RDS instance types. RDS for PostgreSQL was chosen to save on cost.

Technology Stack

This table summarises full tech stack across local development and cloud deployment environments.

Component	Local (Docker compose)	Cloud (AWS)	Purpose
Frontend	Next.js 15 (Node 22)	EKS Deployment + LoadBalancer	User Interface
API Server	FastAPI + Uvicorn (Python 3.12)	EKS Deployment	HTTP Request Handling
Background Worker	RQ Worker (Python 3.12)	EKS Deployment	Async media Processing
Message queue	Redis 7 (Alpine)	EKS Redis Deployment	Job queue/ broker
Database	PostgreSQL 16 (Alpine)	RDS PostgreSQL	Job metadata persistence, User Auth
Object Storage	Local filesystem (/data/media)	Amazon S3	Upload and output files
Container Registry	Local Docker daemon	Amazon ECR	Image storage and versioning

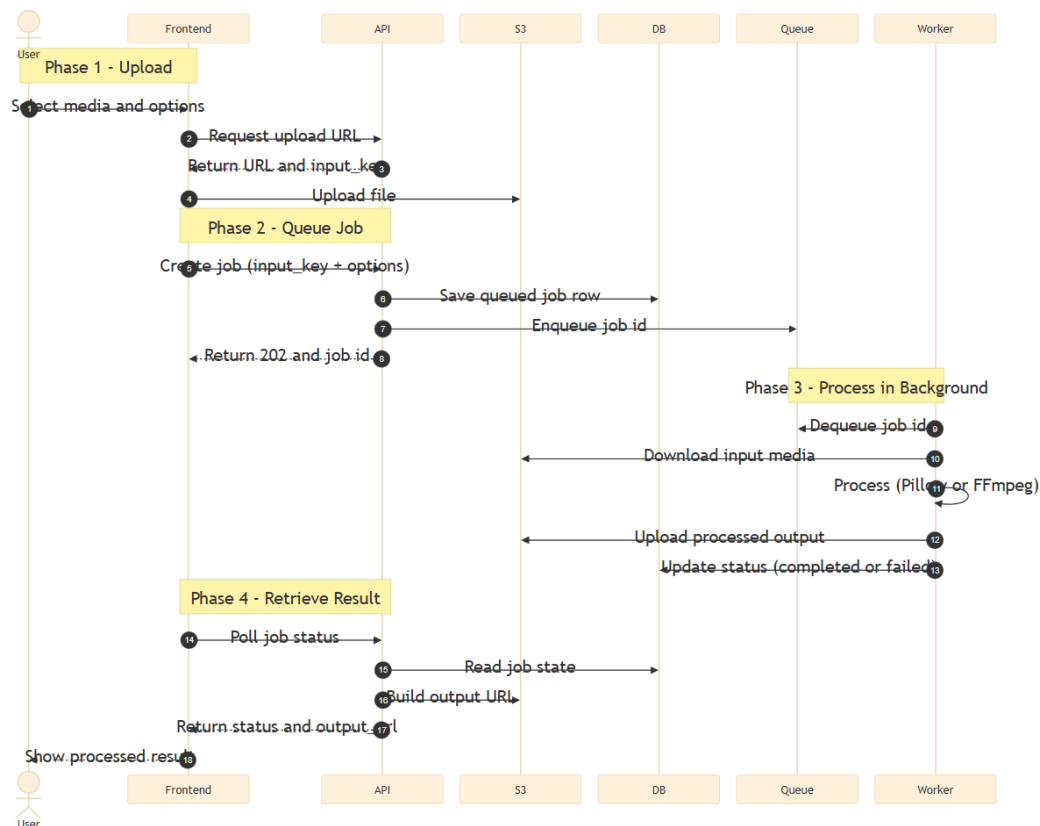
Orchestration	Docker Compose	Amazon EKS (Kubernetes 1.34)	Container scheduling
Image Processing	Pillow 11.1	Pillow 11.1 (in worker pod)	Resize, watermark
DB Migrations	Alembic 1.14	Alembic (kubectl exec)	Schema versioning
Config Management	Dotenv (.env file)	Kubernetes ConfigMap + Secret)	Environment variables

Cloud Architecture:

In the cloud, each Docker Compose maps to Kubernetes workload. The 3 application containers (frontend, api, worker) run as separate Kubernetes Deployments within a media-tool namespace on an EKS cluster provisioned across 2 EC2 instances. Non-application services are replaced by AWS equivalents: PostgreSQL by RDS PostgreSQL, local filesystem by S3 and Redis by AWS k8s Redis container. The container is injected through ConfigMap and two secrets (database URL, JWT Secret , AWS credentials and Redis URL).

Implementation and Deployment:

The end-to-end implementation flow is documented as follows, showing the full asynchronous path from browser request to completed media output.



Cloud Scalability Principles Applied in Implementation:

1. Stateless scale-out at the API tier:
 - The API is intentionally stateless and keeps workflow state in PostgreSQL and Redis.
 - This allows horizontal scaling of API pods without sticky sessions or node-local dependencies.
2. Asynchronous decoupling for elasticity:
 - Media processing is offloaded from request/response paths into a Redis-backed queue and worker tier.
 - This prevents heavy jobs from saturating API capacity and enables independent scaling by queue demand.
3. Independent scaling domains:
 - Frontend, API, and worker run as separate deployable units.
 - Each component can scale based on its own bottleneck profile (traffic, queue depth, or processing time).
4. Automation for repeatable scaling operations:
 - CI/CD builds immutable images, deploys by commit SHA, and validates rollout with smoke tests.
 - This reduces release variance and supports reliable scaling events in cloud environments.

Cloud Deployment (Amazon EKS):

We utilized AWS Pods and Kubernetes Manifests to deploy our backend, frontend, Redis:

- namespace.yaml creates the media-tool namespace, which isolates all workloads and services in one logical application boundary.
- api-deployment.yaml creates the API pod (replicas: 1) with containerPort 8000 and readinessProbe on /health.
- worker-deployment.yaml creates the worker pod (replicas: 1) and starts the RQ worker using `python -m app.worker.runner`.
- frontend-deployment.yaml creates the frontend pod (replicas: 1) and injects `API_INTERNAL_BASE_URL` for in-cluster backend routing.
- redis.yaml creates an in-cluster Redis pod and Service, used as the queue broker between API and worker.
- api Service exposes the API internally on port 8000; frontend Service is type LoadBalancer and publishes the UI externally.

GitHub Workflows (CI/CD to AWS):

We utilized Github workflows CI/CDs to trigger on push to main. “Build and push” configures AWS credentials using OIDC role assumption, logs in to ECR, builds the backend & frontend to push to API and worker repositories with immutable SHA tags. “Redeploy k8s” updates kubeconfig, applies manifests, performs `kubectl set image` to pin commit SHAs, and validates the rollout via health check smoke tests.

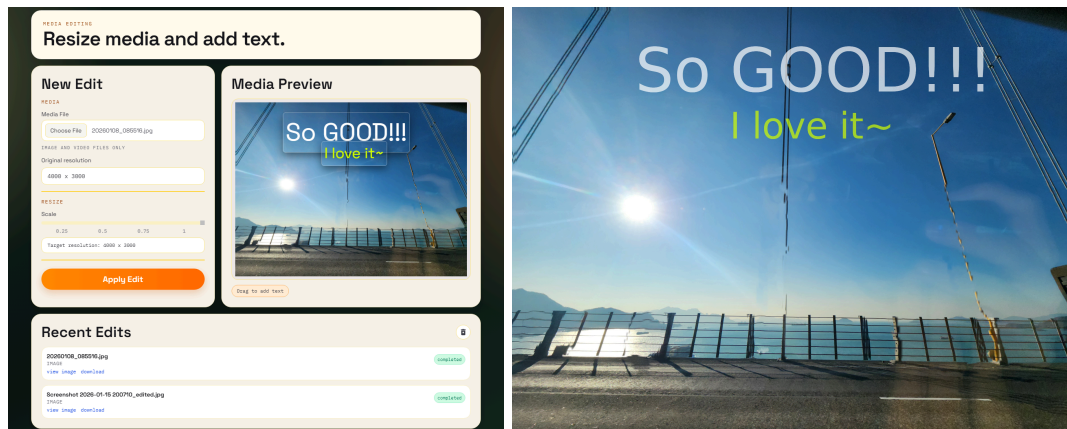
Evaluation:

We executed four focused end-to-end tests on the deployed pipeline to validate media processing correctness.

Test Case	Result	Description
Image - Downscaling	PASS	Test image downscale with lowered resolution target.
Image - Multiple Text Overlays	PASS	Test image edit with multiple text overlays.
Video - Downscaling	PASS	Test video downscale with lowered resolution target.
Video - Multiple Text Overlays	PASS	Test video edit with multiple text overlays.

Result Summary:

- All 4/4 test cases passed.
- The system successfully handled both media types and both transformation modes (downscaling and multi-text overlays) in asynchronous cloud execution.



Conclusion:

This project successfully delivered a fully functional, scalable media processing web application deployed on Amazon EKS. By adopting an asynchronous, event-driven architecture with Redis queues, decoupled workers, and cloud-native storage (S3 + Aurora Serverless), the system overcomes the limitations of traditional synchronous processing, delivering excellent user responsiveness even under load.

The implementation demonstrates core cloud computing principles — stateless design, independent scaling, infrastructure as code, and automated CI/CD — resulting in a production-grade distributed system that is maintainable, observable, and ready for real-world usage.

The live application is accessible [here](#) (note: the URL may update after future redeployments).

Overall, this project provided valuable hands-on experience in building resilient, cloud-native applications on AWS.